TRƯỜNG ĐẠI HỌC BÌNH DƯƠNG
PHÂN HIỆU CÀ MAU
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CUỐI KỲ

HỌC PHẦN: ĐIỆN TOÁN ĐÁM MÂY

ĐỀ TÀI

HỆ THỐNG QUẢN LÝ PHÒNG TRỌ

Ứng dụng web đóng gói và triển khai bằng Docker Compose

ASP.NET Core · PostgreSQL · MinIO

Tên đề tài:	DA05 – Quản Lý Phòng Trọ
Giảng viên hướng dẫn:	Dương Thái Bảo
Nhóm thực hiện:	Nhóm 5
Thành viên:	Trịnh Lâm Huy – 24210501016
	Mã Khánh Tường – 24210501041
	Nguyễn Ca Gu – 24210501053
	Ngô Chí Quyền – 24210501034
	Lê Nhựt Duy – 24210501006

Cà Mau, tháng 10 năm 2026

Lời cảm ơn

Nhóm xin gửi lời cảm ơn chân thành đến giảng viên phụ trách học phần Điện toán đám mây đã truyền đạt nền tảng kiến thức về mô hình dịch vụ, ảo hóa, container và lưu trữ đối tượng, giúp nhóm có đủ cơ sở để thực hiện đồ án này. Nhóm cũng cảm ơn các bạn trong lớp đã góp ý trong quá trình kiểm thử. Do thời gian và kinh nghiệm còn hạn chế, đồ án khó tránh khỏi thiếu sót; nhóm rất mong nhận được ý kiến đóng góp để hoàn thiện hơn.

Danh sách thành viên và phân công nhiệm vụ

STT	Họ và tên	MSSV	Vai trò	Nhiệm vụ chính	Đóng góp
1	Trịnh Lâm Huy	24210501016	Nhóm trưởng, DevOps	Thiết kế kiến trúc, viết Dockerfile, compose.yaml, cấu hình mạng và volume, kiểm chứng dữ liệu bền vững, điều phối tiến độ, video demo	28%
2	Mã Khánh Tường	24210501041	Backend	Thiết kế database, entity, EF Core, REST API cho Phòng, Loại phòng, Người thuê	18%
3	Nguyễn Ca Gu	24210501053	Backend / Storage	Tích hợp MinIO, API upload/download, metadata tệp, API Hợp đồng và Hóa đơn, xác thực – phân quyền	18%
4	Ngô Chí Quyền	24210501034	Frontend	Giao diện Razor + Bootstrap: dashboard, danh sách, form, trang chi tiết, thông báo, hộp thoại xác nhận	18%
5	Lê Nhựt Duy	24210501006	Tester, tài liệu	Kịch bản kiểm thử, Postman Collection, sơ đồ thiết kế, báo cáo, README, slide	18%

Mục lục

Chương 1. Giới thiệu bài toán **1**
1.1. Bối cảnh 1
1.2. Bài toán đặt ra 1
1.3. Phạm vi 1
1.4. Đối tượng sử dụng 1

Chương 2. Mục tiêu hệ thống **2**
2.1. Mục tiêu nghiệp vụ 2
2.2. Mục tiêu kỹ thuật 2
2.3. Mục tiêu học tập 2

Chương 3. Phân tích yêu cầu chức năng và phi chức năng **2**
3.1. Yêu cầu chức năng 2
3.2. Quy tắc nghiệp vụ 3
3.3. Yêu cầu phi chức năng 3

Chương 4. Use Case Diagram **3**
4.1. Tác nhân 3
4.2. Danh sách use case và phân quyền 3
4.3. Mô tả chi tiết một số use case 4

Chương 5. ERD / Class Diagram **5**
5.1. Các thực thể 5
5.2. Quan hệ giữa các thực thể 5

Chương 6. Deployment / Component Diagram **6**
6.1. Mô tả kiến trúc 6
6.2. Luồng dữ liệu 6

Chương 7. Thiết kế database và metadata tệp **7**
7.1. Nguyên tắc thiết kế 7
7.2. Danh sách bảng 8
7.3. Quy ước đặt object key 10
7.4. Chỉ mục và dữ liệu mẫu 10

Chương 8. Thiết kế giao diện web **10**
8.1. Bố cục chung 10
8.2. Danh sách màn hình 10
8.3. Quy tắc trải nghiệm người dùng 11

Chương 9. Thiết kế REST API **11**
9.1. Nguyên tắc 11
9.2. Danh sách endpoint 11
9.3. Mã trạng thái HTTP 12
9.4. Ví dụ yêu cầu và phản hồi 12

Chương 10. Công nghệ sử dụng **12**

Chương 11. Hướng dẫn triển khai Docker **13**
11.1. Cấu trúc thư mục 13
11.2. Dockerfile 13
11.3. Docker Compose 13
11.4. Biến môi trường 15
11.5. Các bước chạy hệ thống 15
11.6. Docker network so với VPC và subnet 15
11.7. Mô hình trách nhiệm chia sẻ 15

Chương 12. Kiểm thử hệ thống **16**
12.1. Môi trường và phương pháp 16
12.2. Ca kiểm thử chức năng 16
12.3. Kiểm chứng dữ liệu bền vững 17

12.4. Kiểm thử bảo mật cơ bản....17
Chương 13. Kết quả đạt được, hạn chế và hướng phát triển....17
13.1. Kết quả đạt được....17
13.2. Hạn chế....18
13.3. Hướng phát triển....18
13.4. Bài học rút ra....18
Tài liệu tham khảo....19
Phụ lục....21
Phụ lục A. Postman Collection....21
Phụ lục B. Đoạn mã upload tệp lên MinIO (rút gọn)....21
Phụ lục C. Ảnh minh chứng....22

Chương 1. Giới thiệu bài toán

1.1. Bối cảnh

Phòng trọ là hình thức lưu trú phổ biến của sinh viên và người lao động tại các đô thị và khu vực gần trường học, khu công nghiệp. Một chủ trọ có thể quản lý từ vài phòng đến vài chục phòng, với nhiều người thuê, nhiều hợp đồng còn hiệu lực và hóa đơn phát sinh hằng tháng (tiền phòng, điện, nước, internet, rác).

Trong thực tế, nhiều chủ trọ vẫn quản lý bằng sổ tay, bảng tính rời rạc hoặc tin nhắn. Cách làm này dẫn đến các vấn đề:

- Khó theo dõi tình trạng phòng (trống, đang thuê, đang sửa chữa) theo thời gian thực.
- Dễ sai sót khi tính tiền điện, nước và tổng hóa đơn mỗi tháng.
- Hợp đồng, ảnh phòng, giấy tờ liên quan lưu phân tán, dễ thất lạc.
- Khó tra cứu lịch sử thuê và lịch sử thanh toán của từng người thuê.
- Dữ liệu không được sao lưu, dễ mất khi thay đổi thiết bị.

1.2. Bài toán đặt ra

Xây dựng một ứng dụng web quản lý phòng trọ cho phép chủ trọ (quản trị viên) và nhân viên quản lý nhập, tra cứu và cập nhật thông tin phòng, người thuê, hợp đồng, hóa đơn; đồng thời lưu trữ tập trung các tệp liên quan như ảnh phòng và bản scan hợp đồng PDF.

Từ góc nhìn của học phần Điện toán đám mây, đồ án không chỉ là một ứng dụng nghiệp vụ mà còn là bài thực hành đóng gói và vận hành một hệ thống nhiều thành phần bằng container: ứng dụng web, cơ sở dữ liệu quan hệ và lưu trữ đối tượng chạy trong các container riêng, nối với nhau qua mạng ảo của Docker Compose và giữ dữ liệu bền vững bằng named volume [1], [2].

1.3. Phạm vi

Trong phạm vi: quản lý loại phòng, phòng, người thuê, hợp đồng, hóa đơn; upload/download tệp lên MinIO; đăng nhập và phân quyền hai vai trò; tìm kiếm; dashboard thống kê cơ bản; API REST kèm Swagger; đóng gói và chạy bằng Docker Compose.

Ngoài phạm vi: thanh toán trực tuyến, HTTPS và tên miền thật, Kubernetes, CI/CD, microservices, ứng dụng di động, nhắc nợ qua SMS. Các nội dung này được nêu ở phần hướng phát triển.

1.4. Đối tượng sử dụng

Đối tượng	Mô tả
Chủ trọ (Administrator)	Quản lý toàn bộ dữ liệu, tài khoản và cấu hình
Nhân viên quản lý (Staff)	Xem dữ liệu, tạo và cập nhật hợp đồng, hóa đơn, tải tệp; không xóa dữ liệu quan trọng

Chương 2. Mục tiêu hệ thống

2.1. Mục tiêu nghiệp vụ

- Số hóa quy trình quản lý phòng trọ từ tiếp nhận người thuê, lập hợp đồng đến thu tiền hằng tháng.
- Giảm sai sót khi tính hóa đơn nhờ công thức được lập trình sẵn.
- Tra cứu nhanh theo tên, mã phòng, số điện thoại hoặc số hợp đồng.
- Lưu trữ tập trung ảnh phòng và hợp đồng PDF, có thể xem hoặc tải xuống bất kỳ lúc nào.

2.2. Mục tiêu kỹ thuật

- Xây dựng kiến trúc nhiều tầng gồm giao diện web, backend, database và object storage.
- Mỗi service có Dockerfile hoặc image riêng; toàn hệ thống khởi động bằng một lệnh docker compose up -d --build.
- Dữ liệu database và tệp trong MinIO tồn tại sau khi chạy docker compose down rồi up lại.
- Cấu hình bí mật quản lý bằng biến môi trường, không đưa mật khẩu thật vào mã nguồn.
- Cung cấp REST API có tài liệu Swagger/OpenAPI và Postman Collection để kiểm thử.

2.3. Mục tiêu học tập

Sinh viên vận dụng các kiến thức về container và image, Compose, volume, mạng ảo, object storage (bucket, object), REST API, CRUD và quản lý cấu hình; đồng thời liên hệ Docker network với khái niệm VPC/subnet và mô hình trách nhiệm chia sẻ đã học ở Bài 1–3.

Chương 3. Phân tích yêu cầu chức năng và phi chức năng

3.1. Yêu cầu chức năng

Mã	Nhóm chức năng	Mô tả	Vai trò
FR01	Xác thực	Đăng nhập, đăng xuất; mật khẩu băm BCrypt	Admin, Staff
FR02	Quản lý loại phòng	Thêm, sửa, xóa, xem danh sách loại phòng (giá niêm yết, diện tích, mô tả)	Admin
FR03	Quản lý phòng	CRUD phòng, tìm theo mã hoặc tên phòng, lọc theo trạng thái, trang chi tiết phòng	Admin, Staff (không xóa)
FR04	Quản lý người thuê	CRUD người thuê (họ tên, CCCD, số điện thoại, quê quán), tìm kiếm	Admin, Staff (không xóa)
FR05	Quản lý hợp đồng	Lập hợp đồng gắn phòng và người thuê, ngày bắt đầu, ngày kết thúc, tiền cọc, giá thuê; kết thúc hợp đồng	Admin, Staff
FR06	Quản lý hóa đơn	Lập hóa đơn tháng theo hợp đồng: chỉ số điện, nước, phí dịch vụ; tự tính tổng; đánh dấu đã thanh toán	Admin, Staff
FR07	Upload tệp	Tải ảnh phòng (JPG, PNG) và hợp đồng scan (PDF) lên MinIO, kiểm tra định dạng và dung lượng	Admin, Staff
FR08	Xem, tải tệp	Hiển thị danh sách tệp theo thực thể; xem ảnh, tải xuống tệp	Admin, Staff
FR09	Dashboard	Thống kê số phòng theo trạng thái, hợp đồng sắp hết hạn, hóa đơn chưa thanh toán	Admin, Staff
FR10	Thông báo	Hiển thị thông báo thành công hoặc lỗi sau mỗi thao tác	Admin, Staff
FR11	REST API	API cho Phòng, Người thuê, Hợp đồng, Hóa đơn, upload/download; Swagger	Admin, Staff
FR12	Quản lý tài khoản	Tạo tài khoản nhân viên, đặt lại mật khẩu, khóa tài khoản	Admin

3.2. Quy tắc nghiệp vụ

1. Mỗi phòng có mã duy nhất (ví dụ P101) và đúng một trạng thái: Trống, Đang thuê hoặc Bảo trì.
2. Một phòng chỉ có tối đa một hợp đồng đang hiệu lực tại một thời điểm.
3. Khi lập hợp đồng, phòng chuyển sang Đang thuê; khi kết thúc hợp đồng, phòng chuyển về Trống.
4. Ngày kết thúc hợp đồng phải sau ngày bắt đầu; giá thuê và tiền cọc không âm.
5. Chỉ số điện, nước kỳ này không nhỏ hơn kỳ trước.
6. Mỗi hợp đồng chỉ có một hóa đơn cho mỗi tháng.
7. Tổng hóa đơn = tiền phòng + (số điện × đơn giá điện) + (số nước × đơn giá nước) + phí dịch vụ.
8. Không xóa người thuê đang có hợp đồng hiệu lực, không xóa phòng đang được thuê.

3.3. Yêu cầu phi chức năng

Mã	Tiêu chí	Yêu cầu
NFR01	Khả năng triển khai	Chạy được bằng một lệnh Compose trên máy cá nhân hoặc máy phòng thực hành có Docker
NFR02	Tính bền vững dữ liệu	Dữ liệu database và MinIO giữ nguyên sau docker compose down và up -d
NFR03	Bảo mật	Mật khẩu băm BCrypt; credential qua biến môi trường; tệp .env không đưa lên Git; frontend không truy cập trực tiếp database hoặc MinIO
NFR04	Hiệu năng	Các trang danh sách phản hồi dưới 2 giây với khoảng 1.000 bản ghi
NFR05	Khả dụng	Giao diện rõ ràng, hiển thị tốt trên màn hình desktop, dùng được trên trình duyệt phổ biến
NFR06	Khả năng bảo trì	Mã nguồn chia lớp (Controller – Service), có README và tài liệu API
NFR07	Tính di động	Cùng một image chạy giống nhau trên Windows, macOS và Linux nhờ container
NFR08	Giới hạn tệp	Tối đa 5 MB cho ảnh, 10 MB cho PDF; kiểm tra phần mở rộng và MIME type

Chương 4. Use Case Diagram

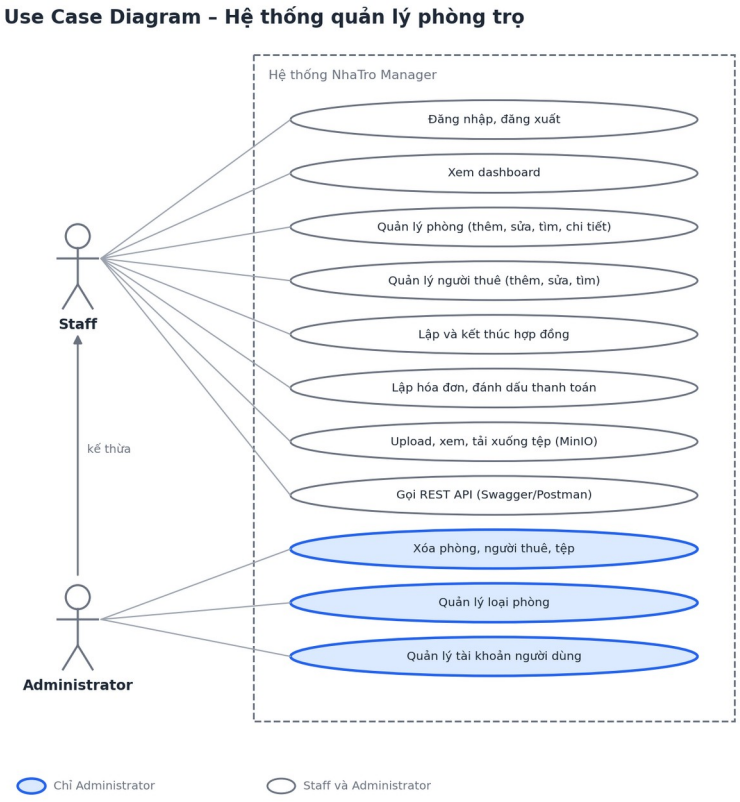
4.1. Tác nhân

Tác nhân	Mô tả
Administrator	Chủ trọ; có toàn quyền trên hệ thống
Staff	Nhân viên quản lý; thao tác nghiệp vụ hằng ngày, bị giới hạn quyền xóa và quản trị tài khoản

4.2. Danh sách use case và phân quyền

Mã	Use case	Administrator	Staff
UC01	Đăng nhập / Đăng xuất	✓	✓
UC02	Xem dashboard	✓	✓
UC03	Quản lý loại phòng	✓	Chỉ xem
UC04	Quản lý phòng (thêm, sửa, tìm kiếm, xem chi tiết)	✓	✓
UC05	Xóa phòng	✓	✗

Mã	Use case	Administrator	Staff
UC06	Quản lý người thuê (thêm, sửa, tìm kiếm)	✓	✓
UC07	Xóa người thuê	✓	✗
UC08	Lập và kết thúc hợp đồng	✓	✓
UC09	Lập hóa đơn và đánh dấu thanh toán	✓	✓
UC10	Upload tệp (ảnh phòng, hợp đồng PDF)	✓	✓
UC11	Xem và tải xuống tệp	✓	✓
UC12	Quản lý tài khoản người dùng	✓	✗
UC13	Gọi REST API qua Swagger/Postman	✓	✓



Hình 4.1. Use Case Diagram

Staff thực hiện nghiệp vụ hằng ngày (bảy nhóm đầu); Administrator kế thừa các quyền đó và có thêm xóa dữ liệu, quản lý loại phòng (Staff chỉ xem) và quản lý tài khoản [3].

4.3. Mô tả chi tiết một số use case

UC08 – Lập hợp đồng thuê phòng

Mục	Nội dung
Tác nhân	Administrator, Staff
Tiền điều kiện	Đã đăng nhập; phòng ở trạng thái Trống; người thuê đã có trong hệ thống
Luồng chính	1) Chọn Thêm hợp đồng. 2) Chọn phòng và người thuê. 3) Nhập ngày bắt đầu, ngày kết thúc, tiền cọc, giá thuê. 4) Hệ thống kiểm tra dữ liệu. 5) Lưu hợp đồng, chuyển phòng sang Đang thuê. 6) Hiển thị thông báo thành công.
Luồng thay thế	Dữ liệu sai (ngày không hợp lệ, phòng đã có hợp đồng): hiển thị lỗi, giữ nguyên form.
Hậu điều kiện	Có hợp đồng mới; trạng thái phòng được cập nhật.

UC10 – Upload tệp

Mục	Nội dung
Tác nhân	Administrator, Staff
Tiền điều kiện	Đã đăng nhập; đang ở trang chi tiết phòng hoặc hợp đồng
Luồng chính	1) Chọn tệp từ máy. 2) Gửi multipart/form-data đến backend. 3) Backend kiểm tra định dạng và dung lượng. 4) Backend lưu tệp vào bucket project-files. 5) Backend lưu metadata vào bảng tep_dinh_kem. 6) Giao diện hiển thị thông báo và danh sách tệp mới.
Luồng thay thế	Sai định dạng hoặc vượt dung lượng: trả lỗi 400 và hiển thị thông báo; không ghi gì vào MinIO.
Hậu điều kiện	Tệp tồn tại trong MinIO; có một bản ghi metadata tương ứng.

UC09 – Lập hóa đơn tháng

Mục	Nội dung
Tác nhân	Administrator, Staff
Tiền điều kiện	Hợp đồng đang hiệu lực; chưa có hóa đơn cho tháng được chọn
Luồng chính	1) Chọn hợp đồng và tháng. 2) Nhập chỉ số điện, nước mới. 3) Hệ thống lấy chỉ số kỳ trước, tính tổng tiền. 4) Lưu hóa đơn ở trạng thái Chưa thanh toán.
Luồng thay thế	Chỉ số mới nhỏ hơn chỉ số cũ hoặc đã tồn tại hóa đơn tháng đó: báo lỗi.
Hậu điều kiện	Hóa đơn được lưu; hiển thị trên dashboard nếu chưa thanh toán.

Chương 5. ERD / Class Diagram

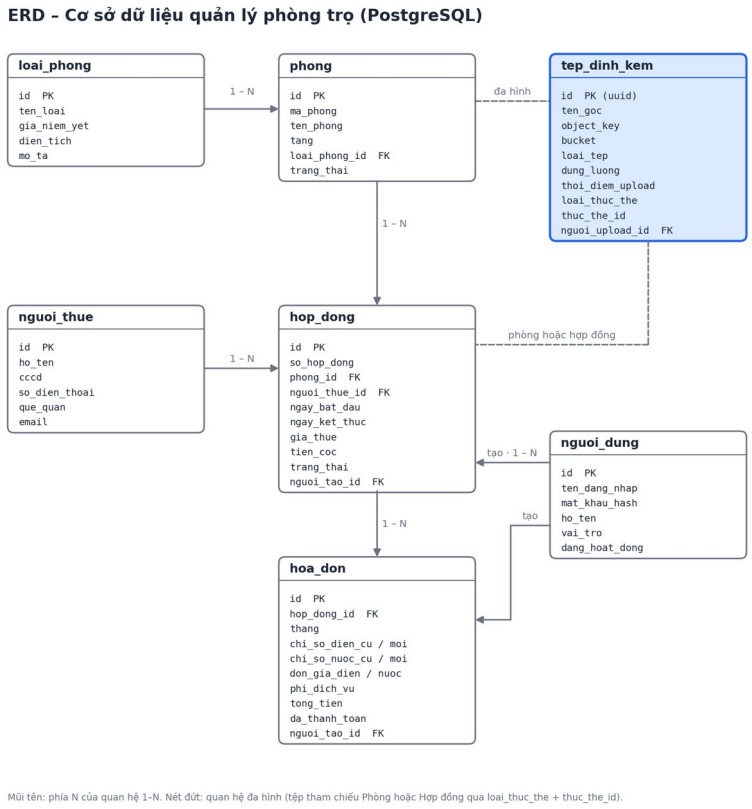
5.1. Các thực thể

Hệ thống gồm bảy thực thể, trong đó có bốn thực thể nghiệp vụ chính (Phòng, Người thuê, Hợp đồng, Hóa đơn): [4]

Thực thể	Ý nghĩa
NguoiDung	Tài khoản đăng nhập và vai trò
LoaiPhong	Phân loại phòng, giá niêm yết, diện tích
Phong	Phòng trọ cụ thể
NguoiThue	Thông tin người thuê
HopDong	Hợp đồng thuê giữa người thuê và phòng
HoaDon	Hóa đơn hằng tháng của một hợp đồng
TepDinhKem	Metadata của tệp được lưu trong MinIO

5.2. Quan hệ giữa các thực thể

- Một LoaiPhong có nhiều Phong; mỗi Phong thuộc một LoaiPhong (1–N).
- Một Phong có nhiều HopDong theo thời gian; mỗi HopDong thuộc một Phong (1–N).
- Một NguoiThue có nhiều HopDong; mỗi HopDong thuộc một NguoiThue (1–N).
- Một HopDong có nhiều HoaDon; mỗi HoaDon thuộc một HopDong (1–N).
- Một NguoiDung tạo nhiều HopDong và HoaDon (1–N, trường người tạo).
- TepDinhKem tham chiếu thực thể liên quan bằng cặp (loại thực thể, ID thực thể), ví dụ ảnh của Phong hoặc bản scan của HopDong (quan hệ đa hình).



Hình 5.1. ERD

Mũi tên là quan hệ 1–N (phía có mũi tên là phía N); nét đứt là quan hệ đa hình: tep_dinh_kem trỏ tới phòng hoặc hợp đồng qua cặp loai_thuc_the, thuc_the_id. Chỉ hiển thị các cột chính; đầy đủ cột xem chương 7.

Chương 6. Deployment / Component Diagram

6.1. Mô tả kiến trúc

Hệ thống chọn **Phương án 1 – giao diện tích hợp backend**: ứng dụng ASP.NET Core vừa phục vụ giao diện Razor View vừa cung cấp REST API. Compose chạy ba service:

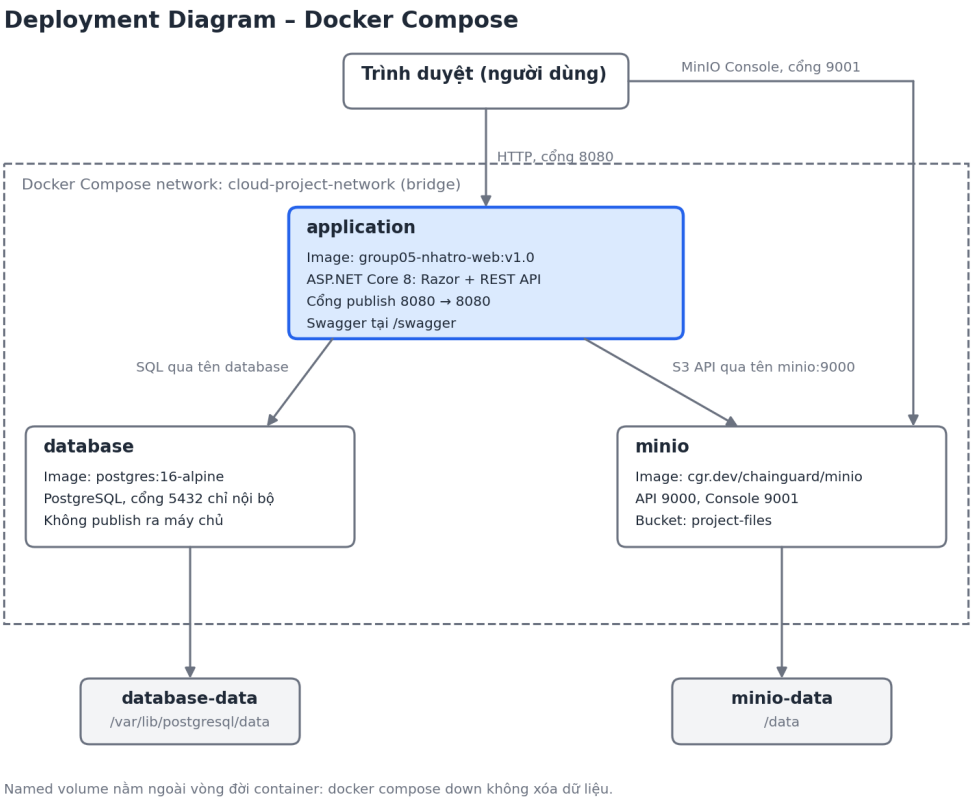
Service	Image / Dockerfile	Vai trò	Cổng (host:container)	Volume
application	Build từ Dockerfile của nhóm (nhatro-web:v1.0)	Giao diện web, REST API, nghiệp vụ	8080:8080	–
database	postgres:16-alpine	Lưu dữ liệu nghiệp vụ và metadata	5432 (chỉ nội bộ)	database-data
minio	cgr.dev/chainguard/minio	Lưu ảnh, PDF	9000:9000 (API), 9001:9001 (Console)	minio-data

Tất cả service thuộc network riêng cloud-project-network (bridge). Chỉ cổng cần demo mới được publish ra máy chủ; database không mở cổng ra ngoài nhằm giảm bề mặt tấn công.

6.2. Luồng dữ liệu

1. Trình duyệt gửi yêu cầu HTTP đến application ở cổng 8080.

2. Application xử lý nghiệp vụ, truy vấn PostgreSQL qua hostname database trong network nội bộ.
3. Khi upload, application gửi tệp đến MinIO qua hostname minio:9000, sau đó ghi metadata vào PostgreSQL.
4. Khi xem hoặc tải tệp, application đọc object từ MinIO và trả về cho trình duyệt (hoặc cấp URL tạm thời có chữ ký). Trình duyệt không bao giờ biết access key của MinIO.



Hình 6.1. Deployment Diagram – Docker Compose

Trình duyệt chỉ nói chuyện với ứng dụng ở cổng 8080 (và MinIO Console ở 9001 khi cần xem bucket); ứng dụng gọi database và MinIO bằng tên service trong network. Hai named volume giữ dữ liệu khi container bị tạo lại.

Chương 7. Thiết kế database và metadata tệp

7.1. Nguyên tắc thiết kế

Cơ sở dữ liệu PostgreSQL chỉ lưu dữ liệu nghiệp vụ và **metadata** của tệp. Nội dung tệp (ảnh, PDF) nằm trong MinIO; bảng tep_dinh_kem chỉ giữ tên gốc, object key, loại tệp, dung lượng và thời điểm upload. Cách tách này giữ cho database nhỏ, sao lưu nhanh và cho phép thay đổi nơi lưu tệp mà không đổi nghiệp vụ [5], [6].

Quy ước: tên bảng và cột viết thường, ngăn cách bằng dấu gạch dưới; khóa chính là id kiểu số nguyên tự tăng (riêng tep_dinh_kem.id dùng UUID); mọi bảng có created_at.

7.2. Danh sách bảng

Bảng nguoi_dung

Cột	Kiểu	Ràng buộc	Mô tả
id	int	PK	Mã tài khoản
ten_dang_nhap	varchar(50)	UNIQUE, NOT NULL	Tên đăng nhập
mat_khau_hash	varchar(100)	NOT NULL	Mật khẩu băm BCrypt
ho_ten	varchar(100)	NOT NULL	Họ tên hiển thị
vai_tro	varchar(20)	NOT NULL	Administrator hoặc Staff
dang_hoat_dong	boolean	NOT NULL	Tài khoản có bị khóa không
created_at	timestamp	NOT NULL	Thời điểm tạo

Bảng loai_phong

Cột	Kiểu	Ràng buộc	Mô tả
id	int	PK	Mã loại phòng
ten_loai	varchar(100)	UNIQUE, NOT NULL	Ví dụ: Phòng đơn, Phòng đôi, Studio
gia_niem_yet	numeric(12,0)	CHECK ≥ 0	Giá thuê gợi ý theo tháng (VNĐ)
dien_tich	numeric(5,1)	CHECK > 0	Diện tích (m²)
mo_ta	text		Mô tả tiện nghi

Bảng phong

Cột	Kiểu	Ràng buộc	Mô tả
id	int	PK	Mã nội bộ
ma_phong	varchar(20)	UNIQUE, NOT NULL	Mã phòng, ví dụ P101
ten_phong	varchar(100)	NOT NULL	Tên hiển thị
tang	int	NOT NULL	Tầng
loai_phong_id	int	FK → loai_phong(id)	Loại phòng
trang_thai	varchar(20)	NOT NULL	Trống / Đang thuê / Bảo trì
ghi_chu	text		Ghi chú

Bảng nguoi_thue

Cột	Kiểu	Ràng buộc	Mô tả
id	int	PK	Mã người thuê
ho_ten	varchar(100)	NOT NULL	Họ tên
cccd	varchar(12)	UNIQUE, NOT NULL	Số căn cước công dân (12 chữ số)
so_dien_thoai	varchar(15)	NOT NULL	Số điện thoại
que_quan	varchar(200)		Quê quán
email	varchar(100)		Email (không bắt buộc)

Bảng hop_dong

Cột	Kiểu	Ràng buộc	Mô tả
id	int	PK	Mã hợp đồng

Cột	Kiểu	Ràng buộc	Mô tả
so_hop_dong	varchar(30)	UNIQUE, NOT NULL	Số hợp đồng
phong_id	int	FK → phong(id)	Phòng được thuê
nguoi_thue_id	int	FK → nguoi_thue(id)	Người đứng tên hợp đồng
ngay_bat_dau	date	NOT NULL	Ngày bắt đầu
ngay_ket_thuc	date	CHECK > ngay_bat_dau	Ngày kết thúc dự kiến
gia_thue	numeric(12,0)	CHECK ≥ 0	Giá thuê thực tế mỗi tháng
tien_coc	numeric(12,0)	CHECK ≥ 0	Tiền đặt cọc
trang_thai	varchar(20)	NOT NULL	Hiệu lực / Đã kết thúc
nguoi_tao_id	int	FK → nguoi_dung(id)	Người lập hợp đồng

Bảng hoa_don

Cột	Kiểu	Ràng buộc	Mô tả
id	int	PK	Mã hóa đơn
hop_dong_id	int	FK → hop_dong(id)	Hợp đồng liên quan
thang	date	NOT NULL	Ngày đầu tháng được tính tiền
chi_so_dien_cu / chi_so_dien_moi	int	CHECK mới ≥ cũ	Chỉ số điện kỳ trước / kỳ này
chi_so_nuoc_cu / chi_so_nuoc_moi	int	CHECK mới ≥ cũ	Chỉ số nước kỳ trước / kỳ này
don_gia_dien, don_gia_nuoc	numeric(10,0)	≥ 0	Đơn giá điện, nước (VNĐ)
phi_dich_vu	numeric(12,0)	≥ 0	Internet, rác, giữ xe
tong_tien	numeric(12,0)	NOT NULL	Tổng cộng theo công thức mục 3.2
da_thanh_toan	boolean	NOT NULL	Trạng thái thanh toán
ngay_thanh_toan	date		Ngày thu tiền
nguoi_tao_id	int	FK → nguoi_dung(id)	Người lập hóa đơn

Ràng buộc duy nhất (hop_dong_id, thang) bảo đảm mỗi hợp đồng chỉ có một hóa đơn mỗi tháng.

Bảng tep_dinh_kem (metadata tệp)

Cột	Kiểu	Ràng buộc	Mô tả
id	uuid	PK	Mã tệp
ten_goc	varchar(255)	NOT NULL	Tên tệp gốc do người dùng chọn
object_key	varchar(300)	UNIQUE, NOT NULL	Đường dẫn trong MinIO, ví dụ uploads/2026/10/3f9c…e1.pdf
bucket	varchar(63)	NOT NULL	Tên bucket (project-files)
loai_tep	varchar(100)	NOT NULL	MIME type (image/jpeg, application/pdf…)
dung_luong	bigint	NOT NULL	Dung lượng (byte)
thoi_diem_upload	timestamp	NOT NULL	Thời điểm upload
loai_thuc_the	varchar(20)	NOT NULL	Phong hoặc HopDong
thuc_the_id	int	NOT NULL	ID của phòng hoặc hợp đồng liên quan
nguoi_upload_id	int	FK → nguoi_dung(id)	Người tải lên

7.3. Quy ước đặt object key

Object key có dạng uploads/{năm}/{tháng}/{uuid}{phần mở rộng}, ví dụ uploads/2026/10/b7e1c2d4-5f6a-4c3b-9e8d-1a2b3c4d5e6f.pdf. Thư mục theo năm và tháng giúp duyệt bucket trong MinIO Console dễ hơn; UUID tránh trùng tên khi nhiều người upload cùng lúc. Tên gốc của tệp chỉ lưu trong database, không đưa vào object key để tránh ký tự đặc biệt và lộ thông tin [6].

7.4. Chỉ mục và dữ liệu mẫu

Chỉ mục được tạo cho các cột tìm kiếm thường dùng: phong(ma_phong), nguoi_thue(ho_ten, cccd, so_dien_thoai), hop_dong(so_hop_dong), tep_dinh_kem(loai_thuc_the, thuc_the_id). Khi khởi động lần đầu, ứng dụng tự tạo bảng (EnsureCreated) và nạp dữ liệu mẫu: hai tài khoản demo, ba loại phòng, mười phòng, năm người thuê, bốn hợp đồng và một số hóa đơn, để giảng viên có thể thao tác ngay [7].

Chương 8. Thiết kế giao diện web

8.1. Bố cục chung

Giao diện dùng Bootstrap 5 và Razor Layout dùng chung gồm: thanh header có tên hệ thống và tài khoản đang đăng nhập; menu điều hướng bên trái (Dashboard, Loại phòng, Phòng, Người thuê, Hợp đồng, Hóa đơn, Tài khoản); vùng nội dung chính; footer ghi tên nhóm và học phần. Người dùng đi giữa các màn hình bằng menu, không phải gõ URL [8], [9].

8.2. Danh sách màn hình

STT	Màn hình	Đường dẫn	Chức năng chính
1	Đăng nhập	/Account/Login	Nhập tên đăng nhập, mật khẩu; báo lỗi khi sai
2	Dashboard	/	Số phòng theo trạng thái, hợp đồng sắp hết hạn trong 30 ngày, hóa đơn chưa thanh toán
3	Danh sách phòng	/Phong	Bảng phòng, ô tìm kiếm, lọc trạng thái, nút Xem, Sửa, Xóa
4	Form thêm / sửa phòng	/Phong/Create, /Phong/Edit/{id}	Nhập mã, tên, tầng, loại, trạng thái; kiểm tra dữ liệu
5	Chi tiết phòng	/Phong/Details/{id}	Thông tin phòng, hợp đồng hiện tại, **ảnh phòng**, nút upload ảnh
6	Danh sách người thuê	/NguoiThue	Bảng, tìm theo tên, CCCD hoặc số điện thoại
7	Form người thuê	/NguoiThue/Create, /Edit/{id}	Thêm, cập nhật người thuê
8	Danh sách hợp đồng	/HopDong	Bảng hợp đồng, tìm theo số hợp đồng hoặc tên người thuê
9	Form hợp đồng	/HopDong/Create	Chọn phòng trống, người thuê, ngày, giá, cọc
10	Chi tiết hợp đồng	/HopDong/Details/{id}	Thông tin hợp đồng, **danh sách hóa đơn**, **bản scan PDF**, nút Kết thúc hợp đồng
11	Danh sách hóa đơn	/HoaDon	Lọc theo tháng và trạng thái thanh toán

STT	Màn hình	Đường dẫn	Chức năng chính
12	Form hóa đơn	/HoaDon/Create	Chọn hợp đồng, nhập chỉ số; tổng tiền hiển thị trước khi lưu
13	Loại phòng, Tài khoản	/LoaiPhong, /TaiKhoan	CRUD cho Administrator

8.3. Quy tắc trải nghiệm người dùng

- Mọi biểu mẫu có nhãn rõ ràng, kiểm tra dữ liệu ở trình duyệt (HTML5) và ở máy chủ (DataAnnotations); thông báo lỗi hiển thị ngay dưới ô nhập.
- Thao tác xóa luôn qua hộp thoại xác nhận (Bootstrap modal) nêu rõ tên đối tượng sắp xóa.
- Sau mỗi thao tác, thông báo thành công (nền xanh) hoặc lỗi (nền đỏ) hiện ở đầu trang và tự đóng sau vài giây.
- Khu vực upload cho phép chọn tệp, hiển thị tên và dung lượng trước khi gửi, báo lỗi nếu sai định dạng hoặc quá dung lượng, rồi hiển thị tệp mới trong danh sách.
- Ảnh phòng hiển thị dạng ảnh thu nhỏ; PDF hiển thị tên, dung lượng và nút Tải xuống.
- Nút và menu bị ẩn với Staff nếu không có quyền (ví dụ nút Xóa), đồng thời backend vẫn kiểm tra quyền để chặn gọi trực tiếp.

Phụ lục ảnh chụp màn hình (thư mục demo/screenshots) minh họa từng màn hình trong bảng 8.2.

Chương 9. Thiết kế REST API

9.1. Nguyên tắc

API nằm dưới tiền tố /api, trao đổi dữ liệu JSON, dùng cùng cơ chế đăng nhập (cookie) với giao diện web. Tài liệu tương tác Swagger UI có tại /swagger. Nhóm đồng thời cung cấp Postman Collection trong source-code/postman/ [10], [11].

9.2. Danh sách endpoint

Nhóm	Phương thức	Đường dẫn	Mô tả	Quyền
Xác thực	POST	/api/auth/login	Đăng nhập	Công khai
Xác thực	POST	/api/auth/logout	Đăng xuất	Đã đăng nhập
Phòng	GET	/api/phong?keyword=&trangThai=	Danh sách, tìm kiếm	Admin, Staff
Phòng	GET	/api/phong/{id}	Chi tiết phòng	Admin, Staff
Phòng	POST	/api/phong	Tạo phòng	Admin, Staff
Phòng	PUT	/api/phong/{id}	Cập nhật phòng	Admin, Staff
Phòng	DELETE	/api/phong/{id}	Xóa phòng	Admin
Người thuê	GET / POST	/api/nguoi-thue	Danh sách (tìm kiếm) / tạo	Admin, Staff
Người thuê	GET / PUT / DELETE	/api/nguoi-thue/{id}	Chi tiết / sửa / xóa	Xóa: Admin
Hợp đồng	GET / POST	/api/hop-dong	Danh sách / lập hợp đồng	Admin, Staff
Hợp đồng	GET / PUT	/api/hop-dong/{id}	Chi tiết / cập nhật	Admin, Staff
Hợp đồng	POST	/api/hop-dong/{id}/ket-thuc	Kết thúc hợp đồng	Admin, Staff

Nhóm	Phương thức	Đường dẫn	Mô tả	Quyền
Hóa đơn	GET / POST	/api/hoa-don	Danh sách / lập hóa đơn	Admin, Staff
Hóa đơn	PUT	/api/hoa-don/{id}/thanh-toan	Đánh dấu đã thanh toán	Admin, Staff
Tệp	POST	/api/tep? loaiThucThe=&thucTheId=	Upload (multipart/form-data)	Admin, Staff
Tệp	GET	/api/tep? loaiThucThe=&thucTheId=	Danh sách tệp của một thực thể	Admin, Staff
Tệp	GET	/api/tep/{id}/download	Tải hoặc xem tệp	Admin, Staff
Tệp	DELETE	/api/tep/{id}	Xóa tệp (object và metadata)	Admin

9.3. Mã trạng thái HTTP

Tình huống	Mã	Ví dụ trong hệ thống
Lấy dữ liệu thành công	200 OK	GET danh sách phòng
Tạo thành công	201 Created	POST phòng, hợp đồng, upload tệp; kèm header Location
Cập nhật hoặc xóa thành công	204 No Content	PUT, DELETE
Dữ liệu không hợp lệ	400 Bad Request	Ngày kết thúc trước ngày bắt đầu; tệp sai định dạng; chỉ số điện giảm
Chưa đăng nhập	401 Unauthorized	Gọi API khi chưa đăng nhập
Không đủ quyền	403 Forbidden	Staff gọi DELETE phòng
Không tìm thấy	404 Not Found	Phòng, hợp đồng hoặc tệp không tồn tại
Xung đột nghiệp vụ	409 Conflict	Mã phòng trùng; phòng đã có hợp đồng hiệu lực
Lỗi hệ thống	500 Internal Server Error	Lỗi không lường trước, đã được ghi log

Mã 409 là bổ sung của nhóm cho các xung đột nghiệp vụ. Nội dung lỗi trả về theo định dạng ProblemDetails (type, title, status, detail) [12].

9.4. Ví dụ yêu cầu và phản hồi

Tạo hợp đồng:

```
POST /api/hop-dong
{
  "soHopDong": "HD-2026-0001",
  "phongId": 3,
  "nguoiThueId": 2,
  "ngayBatDau": "2026-10-01",
  "ngayKetThuc": "2027-03-31",
  "giaThue": 2500000,
  "tienCoc": 2500000
}
```

Phản hồi 201 Created kèm hợp đồng vừa tạo. Nếu phòng đã có hợp đồng hiệu lực, hệ thống trả 409 Conflict với detail giải thích lý do.

Chương 10. Công nghệ sử dụng

Thành phần	Công nghệ	Lý do chọn
Backend và giao diện	ASP.NET Core 8 (MVC + Web API), C#	Một ứng dụng phục vụ cả giao diện Razor và REST API, phù hợp phương án 1 của đề

Thành phần	Công nghệ	Lý do chọn
ORM	Entity Framework Core với Npgsql	Ánh xạ bảng, truy vấn LINQ, giảm mã SQL thủ công
Database	PostgreSQL 16	Mã nguồn mở, ổn định, có image chính thức
Object storage	MinIO (tương thích giao thức S3)	Chạy cục bộ trong container; dùng cùng khái niệm bucket và object như các dịch vụ lưu trữ đối tượng trên cloud
Frontend	Razor View, Bootstrap 5, JavaScript thuần	Không cần bước build riêng, dễ demo
Bảo mật	BCrypt.Net-Next, Cookie Authentication, phân quyền theo vai trò	Mật khẩu không lưu plaintext; không cần JWT theo đề
Tài liệu API	Swashbuckle (Swagger/OpenAPI), Postman	Kiểm thử API trực quan
Đóng gói	Docker, Docker Compose	Yêu cầu bắt buộc
Quản lý mã nguồn	Git, GitHub	Làm việc nhóm, lưu lịch sử

Chương 11. Hướng dẫn triển khai Docker

11.1. Cấu trúc thư mục

```
source-code/
├── backend/       # ASP.NET Core: Controllers, Services, Models, Views (Razor),
wwwroot
├── frontend/      # Ghi chú: giao diện tích hợp trong backend/Views
├── postman/       # Postman Collection
├── Dockerfile
├── compose.yaml
├── .env.example
├── .gitignore
└── README.md
```

11.2. Dockerfile

Dockerfile dùng build nhiều giai đoạn: giai đoạn build dùng SDK để biên dịch, giai đoạn cuối chỉ chứa runtime nên image nhỏ và an toàn hơn [13].

```
# ---- Giai đoạn build ----
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY backend/*.csproj ./
RUN dotnet restore
COPY backend/ ./
RUN dotnet publish -c Release -o /app/publish --no-restore

# ---- Giai đoạn chạy ----
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
ENTRYPOINT ["dotnet", "NhaTro.Web.dll"]
```

Image của nhóm được đặt tên group05-nhatro-web:v1.0.

11.3. Docker Compose

```
services:
  application:
    build: .
    image: group05-nhatro-web:v1.0
    container_name: nhatro-web
```

```
    ports:
      - "8080:8080"
    environment:
      ConnectionStrings__Default: "Host=database;Port=5432;Database=$
{DB_NAME};Username=${DB_USER};Password=${DB_PASSWORD}"
      Minio__Endpoint: "minio:9000"
      Minio__AccessKey: ${MINIO_ROOT_USER}
      Minio__SecretKey: ${MINIO_ROOT_PASSWORD}
      Minio__Bucket: project-files
    depends_on:
      database:
        condition: service_healthy
      minio:
        condition: service_started
    networks: [cloud-project-network]
    restart: unless-stopped

  database:
    image: postgres:16-alpine
    container_name: nhatro-db
    environment:
      POSTGRES_DB: ${DB_NAME}
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - database-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 5s
      timeout: 5s
      retries: 10
    networks: [cloud-project-network]
    restart: unless-stopped

  minio:
    image: cgr.dev/chainguard/minio:latest
    container_name: nhatro-minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER}
      MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD}
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - minio-data:/data
    networks: [cloud-project-network]
    restart: unless-stopped

volumes:
  database-data:
  minio-data:

networks:
  cloud-project-network:
    driver: bridge
```

Cơ sở dữ liệu không publish cổng ra máy chủ; chỉ service application truy cập được qua hostname database trong network. Ứng dụng tự tạo bucket project-files khi khởi động nếu chưa tồn tại [14].

Lưu ý về image MinIO: từ năm 2026, MinIO không còn phát hành image bản community trên Docker Hub (minio/minio) nên lệnh docker pull minio/minio báo lỗi. Nhóm dùng image cgr.dev/chainguard/minio được build từ mã nguồn MinIO; image này chạy cùng lệnh server /data --console-address ":9001" và cùng biến MINIO_ROOT_USER, MINIO_ROOT_PASSWORD. Nếu máy phòng thực hành không pull được image này,

phương án dự phòng là tự build image từ mã nguồn MinIO bằng Dockerfile của dự án MinIO, hoặc dùng một object storage tương đương (đề cho phép). Nhóm cần chạy thử docker compose pull trước buổi demo.

11.4. Biến môi trường

Tệp .env.example chỉ chứa giá trị mẫu; mỗi thành viên sao chép thành .env và đổi mật khẩu trước khi chạy. Tệp .env nằm trong .gitignore và không được nộp hoặc đưa lên GitHub.

```
DB_NAME=projectdb
DB_USER=projectuser
DB_PASSWORD=change_this_password
MINIO_ROOT_USER=minioadmin
MINIO_ROOT_PASSWORD=change_this_password
```

11.5. Các bước chạy hệ thống

1. Cài Docker Desktop hoặc Docker Engine (có Docker Compose).
2. Sao chép .env.example thành .env và đặt mật khẩu.
3. Chạy docker compose up -d --build tại thư mục source-code.
4. Mở giao diện web: http://localhost:8080; Swagger: http://localhost:8080/swagger; MinIO Console: http://localhost:9001.
5. Đăng nhập bằng tài khoản demo trong README (ví dụ admin / Admin@123 và staff / Staff@123, nhóm đổi theo thực tế).
6. Dừng hệ thống: docker compose down. Xóa toàn bộ dữ liệu demo: docker compose down -v (chỉ dùng khi muốn làm sạch, không dùng khi kiểm chứng dữ liệu bền vững).

11.6. Docker network so với VPC và subnet

Network cloud-project-network đóng vai trò tương tự một VPC thu nhỏ trên máy cá nhân: [14], [15]

Khái niệm cloud (Bài 3)	Tương ứng trong đồ án
VPC: mạng riêng ảo cô lập	Docker bridge network cloud-project-network
Subnet riêng (private)	Database chỉ nằm trong network, không publish cổng ra ngoài
Subnet công khai	Service application và MinIO Console có publish cổng để demo
Security group / quy tắc truy cập	Cổng được publish là cổng duy nhất tiếp cận từ trình duyệt; container khác chỉ gọi nhau qua tên service
DNS nội bộ	Docker phân giải tên database, minio thành địa chỉ IP container

Điểm khác biệt: Docker network không có bảng định tuyến, NAT gateway hay nhiều vùng sẵn sàng như VPC thật, và nhóm không cấu hình tường lửa chi tiết. Đây là mô hình cô lập mức cơ bản, đủ để minh họa nguyên tắc tách tầng.

11.7. Mô hình trách nhiệm chia sẻ

Khi chạy trên máy cá nhân, nhóm tự chịu trách nhiệm cho toàn bộ chồng hạ tầng: phần cứng, hệ điều hành, Docker, cập nhật image, sao lưu volume, mật khẩu và an toàn dữ liệu. Với dịch vụ cloud được quản lý (ví dụ cơ sở dữ liệu hoặc lưu trữ đối tượng do nhà cung cấp vận hành), nhà cung cấp lo phần hạ tầng và nền tảng bên dưới, khách hàng vẫn chịu trách nhiệm về cấu

hình, quyền truy cập và dữ liệu của mình. Đồ án cho nhóm trải nghiệm phần trách nhiệm lớn hơn so với dùng dịch vụ quản lý sẵn [16], [17].

Chương 12. Kiểm thử hệ thống

12.1. Môi trường và phương pháp

Kiểm thử thủ công trên Docker Desktop, trình duyệt Chrome, kết hợp Swagger và Postman cho API. Mỗi ca kiểm thử có dữ liệu đầu vào, kết quả mong đợi; cột **Kết quả thực tế** do nhóm điền sau khi chạy và đính kèm ảnh chụp ở phụ lục.

12.2. Ca kiểm thử chức năng

Mã	Chức năng	Bước thực hiện	Kết quả mong đợi	Kết quả thực tế
TC01	Đăng nhập đúng	Nhập admin và mật khẩu đúng	Vào Dashboard, hiển thị tên người dùng	
TC02	Đăng nhập sai	Nhập sai mật khẩu	Thông báo lỗi, ở lại trang đăng nhập	
TC03	Truy cập khi chưa đăng nhập	Gọi /Phong hoặc GET /api/phong	Chuyển về đăng nhập (web) hoặc 401 (API)	
TC04	Thêm phòng	Nhập P201, tầng 2, chọn loại phòng	Lưu thành công, phòng hiện trong danh sách	
TC05	Thêm phòng trùng mã	Nhập lại mã P201	Báo lỗi trùng mã (409 với API)	
TC06	Validation phòng	Để trống mã phòng	Báo lỗi dưới ô nhập, không lưu	
TC07	Cập nhật phòng	Sửa trạng thái sang Bảo trì	Danh sách hiển thị trạng thái mới	
TC08	Xóa phòng (Admin)	Bấm Xóa, xác nhận	Phòng biến mất khỏi danh sách	
TC09	Xóa phòng (Staff)	Đăng nhập Staff, gọi DELETE /api/phong/{id}	403 Forbidden; nút Xóa không hiển thị	
TC10	Tìm kiếm phòng	Gõ "P20" vào ô tìm kiếm	Chỉ hiện các phòng có mã hoặc tên khớp	
TC11	Thêm người thuê	Nhập đủ họ tên, CCCD 12 số, số điện thoại	Lưu thành công	
TC12	Validation CCCD	Nhập CCCD 9 số	Báo lỗi định dạng	
TC13	Lập hợp đồng	Chọn phòng trống, người thuê, ngày, giá	Hợp đồng được lưu, phòng chuyển Đang thuê	
TC14	Ngày hợp đồng sai	Ngày kết thúc trước ngày bắt đầu	Báo lỗi 400, không lưu	
TC15	Phòng đã có hợp đồng	Lập hợp đồng thứ hai cho cùng phòng	Báo lỗi 409	
TC16	Kết thúc hợp đồng	Bấm Kết thúc hợp đồng	Hợp đồng Đã kết thúc, phòng về Trống	
TC17	Lập hóa đơn	Nhập chỉ số điện, nước mới	Tổng tiền đúng công thức mục 3.2	
TC18	Chỉ số giảm	Nhập chỉ số điện nhỏ hơn kỳ trước	Báo lỗi, không lưu	
TC19	Hóa đơn trùng tháng	Lập hóa đơn thứ hai cùng tháng	Báo lỗi 409	

Mã	Chức năng	Bước thực hiện	Kết quả mong đợi	Kết quả thực tế
TC20	Upload ảnh hợp lệ	Chọn ảnh JPG dưới 5 MB ở chi tiết phòng	Thông báo thành công; ảnh hiện trong danh sách; object có trong MinIO; có bản ghi metadata	
TC21	Upload PDF hợp đồng	Chọn PDF dưới 10 MB ở chi tiết hợp đồng	Như TC20	
TC22	Upload sai định dạng	Chọn tệp .exe hoặc .zip	Báo lỗi 400, không có object mới	
TC23	Upload quá dung lượng	Chọn ảnh lớn hơn 5 MB	Báo lỗi, không có object mới	
TC24	Tải xuống tệp	Bấm Tải xuống trên PDF	Trình duyệt tải đúng tệp, tên gốc được giữ	
TC25	Xem ảnh	Mở ảnh trong chi tiết phòng	Ảnh hiển thị đúng	
TC26	Swagger	Mở /swagger, chạy GET phòng	Trả 200 và dữ liệu JSON	

12.3. Kiểm chứng dữ liệu bền vững

Các bước thực hiện đúng theo yêu cầu của đề, không dùng docker compose down -v:

Bước	Lệnh hoặc thao tác	Bằng chứng cần chụp
1	docker compose up -d --build	Kết quả docker compose ps với ba service đang chạy
2	Tạo một phòng và một hợp đồng qua giao diện	Danh sách có bản ghi mới
3	Upload một ảnh phòng và một PDF hợp đồng	Danh sách tệp; MinIO Console hiển thị object trong bucket project-files
4	docker compose down	Container đã dừng; docker volume ls vẫn còn database-data và minio-data
5	docker compose up -d	Ba service chạy lại
6	Đăng nhập lại, mở danh sách phòng, hợp đồng và chi tiết phòng	Bản ghi vẫn còn; ảnh và PDF vẫn xem và tải được; MinIO Console vẫn thấy object

Hệ thống bền vững vì cả hai dịch vụ lưu dữ liệu vào named volume nằm ngoài vòng đời container: docker compose down chỉ xóa container và network, không xóa volume [18].

12.4. Kiểm thử bảo mật cơ bản

- Mật khẩu trong bảng nguoi_dung là chuỗi băm BCrypt, không đọc ngược được [19].
- docker compose config và kho mã nguồn không chứa mật khẩu thật; .env bị bỏ qua bởi Git.
- Trình duyệt chỉ giao tiếp với cổng 8080; không truy cập trực tiếp được PostgreSQL (cổng 5432 không publish).
- Link tải tệp đi qua backend sau khi kiểm tra đăng nhập, không lộ khóa truy cập MinIO.

Chương 13. Kết quả đạt được, hạn chế và hướng phát triển

13.1. Kết quả đạt được

Yêu cầu của đề	Cách hệ thống đáp ứng
Giao diện web, menu, dashboard	Razor + Bootstrap, menu bên trái, dashboard thống kê

Yêu cầu của đề	Cách hệ thống đáp ứng
Ít nhất 4 thực thể, CRUD cho 3 thực thể	7 bảng; CRUD cho Phòng, Người thuê, Hợp đồng, Hóa đơn, Loại phòng
Validation, tìm kiếm, trang chi tiết	DataAnnotations; tìm theo mã, tên, CCCD, số điện thoại; chi tiết phòng và hợp đồng
Upload và download qua MinIO	Bucket project-files; metadata trong tep_dinh_kem; kiểm tra loại và dung lượng
REST API, Swagger, mã HTTP	Tiền tố /api, Swagger UI, Postman, mã 200/201/204/400/401/403/404/409/500
Phân quyền hai vai trò	Administrator và Staff; mật khẩu BCrypt
Dockerfile, Compose, volume, network, biến môi trường	Dockerfile nhiều giai đoạn; compose.yaml ba service; hai named volume; network riêng; .env.example
Dữ liệu bền vững	Kiểm chứng theo mục 12.3
Sơ đồ và README	Use Case, ERD, Deployment; README đủ 11 mục

13.2. Hạn chế

- Chưa có phân trang; số lượng bản ghi lớn có thể làm danh sách chậm (đề không yêu cầu).
- Chưa dùng HTTPS và tên miền; thông tin đăng nhập truyền qua HTTP trong mạng cục bộ.
- Xác thực bằng cookie, chưa có đặt lại mật khẩu qua email.
- Hóa đơn tính theo đơn giá nhập tay, chưa có bậc thang giá điện.
- Sao lưu volume thực hiện thủ công.

• Đã bổ sung CSS responsive; cần kiểm thử thêm trên thiết bị thực tế.

13.3. Hướng phát triển

- Gửi hóa đơn PDF và nhắc nợ qua email hoặc Zalo.
- Cổng thông tin cho người thuê xem hóa đơn và gửi yêu cầu sửa chữa.
- Báo cáo doanh thu và công nợ theo tháng.
- Triển khai lên máy ảo cloud (Oracle Cloud Free Tier hoặc Azure for Students) với cùng compose.yaml, thêm HTTPS qua reverse proxy.
- Tự động sao lưu volume và đồng bộ lên object storage khác.
- Bổ sung CI/CD để tự kiểm thử và build image.

13.4. Bài học rút ra

Qua đồ án, nhóm hiểu rõ hơn vai trò của container trong việc làm cho môi trường chạy lặp lại được, sự khác nhau giữa dữ liệu trong container và dữ liệu trong volume, và cách object storage tách việc lưu tệp khỏi cơ sở dữ liệu. Nhóm cũng thực hành làm việc theo phân công, dùng Git và viết tài liệu kỹ thuật.

Tài liệu tham khảo

[1] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-145, Sep. 2011. doi: 10.6028/NIST.SP.800-145.

[2] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, Mar. 2014, Art. no. 2.

[3] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2003.

[4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[5] PostgreSQL Global Development Group, "PostgreSQL 16 documentation." [Online]. Available: https://www.postgresql.org/docs/16/ (accessed Oct. 27, 2026).

[6] MinIO, Inc., "Core administration concepts," MinIO Object Store documentation. [Online]. Available: https://docs.min.io/community/minio-object-store/administration/concepts.html (accessed Oct. 27, 2026).

[7] Microsoft, "Entity Framework Core documentation," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/ef/core/ (accessed Oct. 27, 2026).

[8] Bootstrap team, "Bootstrap 5.3 documentation." [Online]. Available: https://getbootstrap.com/docs/5.3/ (accessed Oct. 27, 2026).

[9] S. Smith, "Overview of ASP.NET Core MVC," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/aspnet/core/mvc/overview (accessed Oct. 27, 2026).

[10] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000. [Online]. Available: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (accessed Oct. 27, 2026).

[11] OpenAPI Initiative, "OpenAPI Specification v3.0.3." [Online]. Available: https://spec.openapis.org/oas/v3.0.3.html (accessed Oct. 27, 2026).

[12] R. Fielding, M. Nottingham, and J. Reschke, Eds., "HTTP semantics," Internet Engineering Task Force, RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/info/rfc9110 (accessed Oct. 27, 2026).

[13] Docker Inc., "Multi-stage builds," Docker Docs. [Online]. Available: https://docs.docker.com/build/building/multi-stage/ (accessed Oct. 27, 2026).

[14] Docker Inc., "Networking in Compose," Docker Docs. [Online]. Available: https://docs.docker.com/compose/how-tos/networking/ (accessed Oct. 27, 2026).

[15] Amazon Web Services, "How Amazon VPC works," Amazon VPC User Guide. [Online]. Available: https://docs.aws.amazon.com/vpc/latest/userguide/how-it-works.html (accessed Oct. 27, 2026).

[16] Amazon Web Services, "Shared responsibility model," AWS Cloud Security. [Online]. Available: https://aws.amazon.com/compliance/shared-responsibility-model/ (accessed Oct. 27, 2026).

[17] T. Erl, Z. Mahmood, and R. Puttini, *Cloud Computing: Concepts, Technology & Architecture*. Upper Saddle River, NJ, USA: Prentice Hall, 2013.

[18] Docker Inc., "Volumes," Docker Docs. [Online]. Available: https://docs.docker.com/engine/storage/volumes/ (accessed Oct. 27, 2026).

[19] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf. (FREENIX Track)*, Monterey, CA, USA, Jun. 1999, pp. 81–92.

Phụ lục

Phụ lục A. Postman Collection

Tệp postman/NhaTro.postman_collection.json gồm các nhóm yêu cầu: Auth, Phong, NguoiThue, HopDong, HoaDon, Tep. Biến baseUrl mặc định http://localhost:8080. Thứ tự gợi ý: đăng nhập, tạo phòng, tạo người thuê, lập hợp đồng, lập hóa đơn, upload tệp, tải tệp.

Phụ lục B. Đoạn mã upload tệp lên MinIO (rút gọn)

```
public async Task<(Kq kq, TepDinhKem tep)> UploadAsync(IFormFile file, string
loaiThucThe, int thucTheId, int? userId)
{
    if (file == null || file.Length == 0) return (Kq.That("Vui lòng chọn tệp để tải
lên."), null);

    var contentType = (file.ContentType ?? "").ToLowerInvariant();
    var tenGoc = Path.GetFileName(file.FileName ?? "");
    var ext = Path.GetExtension(tenGoc).ToLowerInvariant();
    if (!ChoPhep.TryGetValue(contentType, out var exts) || !exts.Contains(ext))
        return (Kq.That("Chỉ chấp nhận ảnh JPG, PNG hoặc tệp PDF."), null);

    var gioiHan = contentType == "application/pdf" ? MaxPdf : MaxAnh;
    if (file.Length > gioiHan)
        return (Kq.That($"Tệp vượt quá dung lượng cho phép ({gioiHan / 1024 / 1024}
MB)."), null);
    if (tenGoc.Length > 255) tenGoc = tenGoc.Substring(tenGoc.Length - 255);

    bool tonTai;
    if (loaiThucThe == "Phong") tonTai = await _db.Phongs.AnyAsync(p => p.Id ==
thucTheId);
    else if (loaiThucThe == "HopDong") tonTai = await _db.HopDongs.AnyAsync(h => h.Id
== thucTheId);
    else return (Kq.That("Loại thực thể không hợp lệ (Phong hoặc HopDong)."), null);
    if (!tonTai) return (Kq.That("Không tìm thấy phòng hoặc hợp đồng cần đính kèm
tệp.", 404), null);

    await EnsureBucketAsync();

    var id = Guid.NewGuid();
    var now = DateTime.Now;
    var key = "uploads/" + now.ToString("yyyy") + "/" + now.ToString("MM") + "/" + id
+ ext;

    using (var stream = file.OpenReadStream())
    {
        await _minio.PutObjectAsync(new PutObjectArgs()
            .WithBucket(_bucket)
            .WithObject(key)
            .WithStreamData(stream)
            .WithObjectSize(file.Length)
            .WithContentType(contentType));
    }

    var tep = new TepDinhKem
    {
        Id = id, TenGoc = tenGoc, ObjectKey = key, Bucket = _bucket, LoaiTep =
contentType,
        DungLuong = file.Length, ThoiDiemUpload = now, LoaiThucThe = loaiThucThe,
        ThucTheId = thucTheId, NguoiUploadId = userId
    };
    _db.TepDinhKems.Add(tep);
    try
    {
        await _db.SaveChangesAsync();
    }
```

```
    catch
    {
        await XoaObjectAsync(_bucket, key);
        throw;
    }
    return (Kq.Thanh(), tep);
}
```

Phụ lục C. Ảnh minh chứng

Ảnh chụp màn hình các chức năng, kết quả docker compose ps, MinIO Console và các bước kiểm chứng dữ liệu bền vững được lưu trong demo/screenshots/ và dán vào báo cáo bản PDF cuối cùng.